

Desarrollo e Implementación de un Sistema Cliente-Servidor para el Control Remoto de un Robot con 3 Grados de Libertad

Programación Orientada a Objetos

**Trabajo Práctico Integrador
Ingeniería en Mecatrónica
Universidad Nacional de Cuyo**

Integrantes:

Francisco Castel - 13784

Juan Ignacio Quiroga - 13889

Matías Armani - 13664

Profesores:

Ing. César Omar Aranda

Ing. Facundo Martín

30 de octubre de 2024

Índice

1. Introducción	3
1.1. Descripción general del problema	3
1.2. Objetivo del Proyecto	3
2. Desarrollo del Proyecto	3
2.1. Fundamentos y esquema descriptivo	3
2.2. Descripción de los componentes software	5
2.3. Instalación, vinculación y uso del sistema	6
2.3.1. Instalación del servidor	6
2.3.2. Instalación del cliente	6
2.4. Diagrama de Secuencia	6
2.5. Diagrama de Actividad	7
2.6. Detalles de instalación, configuración y ejecución	9
2.7. Capturas de Pantalla	9
2.8. Pruebas Unitarias	12
3. Disposición de directorios	13
4. Conclusiones	14
4.1. Dificultades enfrentadas	14
4.2. Bugs	14
4.3. Ventajas y desventajas de los componentes utilizados	14
4.4. Posibles extensiones	15
5. Referencias	15
5.1. Bibliografía y sitios web consultados	15
5.2. Recursos adicionales consultados	15

1. Introducción

1.1. Descripción general del problema

El presente proyecto desarrolla una solución para el control remoto de un robot de tres grados de libertad (RRR), abordando las complejidades de comunicación entre un cliente y un servidor en diferentes lenguajes de programación: C++ y Python. La solución responde a la necesidad de una gestión precisa y eficiente de comandos en el robot, los cuales se ejecutan de forma remota a través del protocolo XML-RPC. Este tipo de arquitectura permite un enfoque modular y escalable, facilitando futuras expansiones y adaptaciones del sistema en entornos industriales o educativos.

1.2. Objetivo del Proyecto

El objetivo es la creación de un sistema cliente-servidor que controle de forma remota los movimientos de un robot con arquitectura RRR, empleando principios avanzados de Programación Orientada a Objetos (POO) y la tecnología XML-RPC. Con esta aplicación, se busca mejorar la precisión y eficiencia en el control de robots, simplificando la transmisión y gestión de comandos, y optimizando el flujo de datos entre cliente y servidor. Además, se pretende proporcionar una base robusta para el control de robots complejos y configurar una estructura fácilmente adaptable para futuras expansiones, como el uso de una interfaz gráfica.

2. Desarrollo del Proyecto

2.1. Fundamentos y esquema descriptivo

El sistema se estructura en una arquitectura cliente-servidor, separando la lógica de control (en el servidor, desarrollado en Python) y la interfaz de usuario (en el cliente, en C++). Se emplea el protocolo XML-RPC, que facilita la comunicación remota utilizando HTTP como medio de transmisión y XML para codificar datos. Este protocolo resulta óptimo para sistemas con requerimientos de bajo overhead en la comunicación, proporcionando un enfoque eficiente y sencillo de implementar, en comparación con alternativas como SOAP.

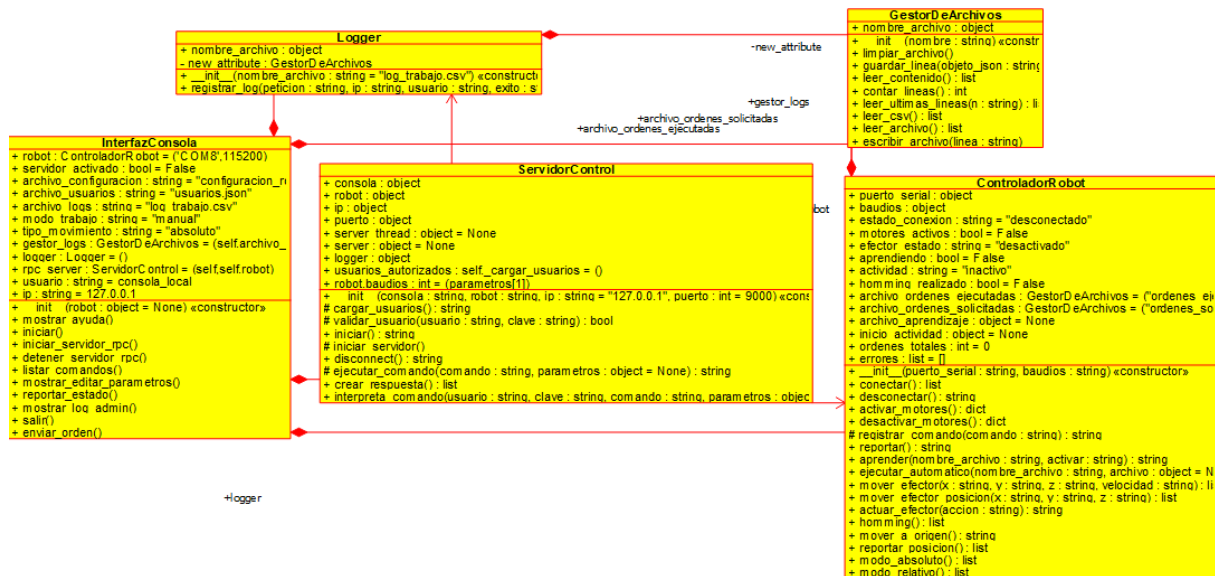


Figura 1: Esquema general del servidor

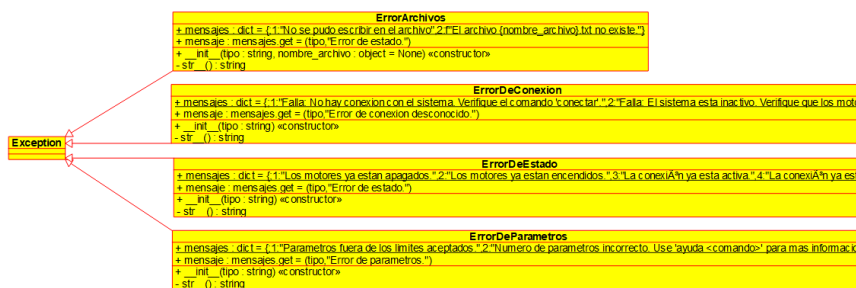


Figura 2: Esquema de manejo de errores del servidor

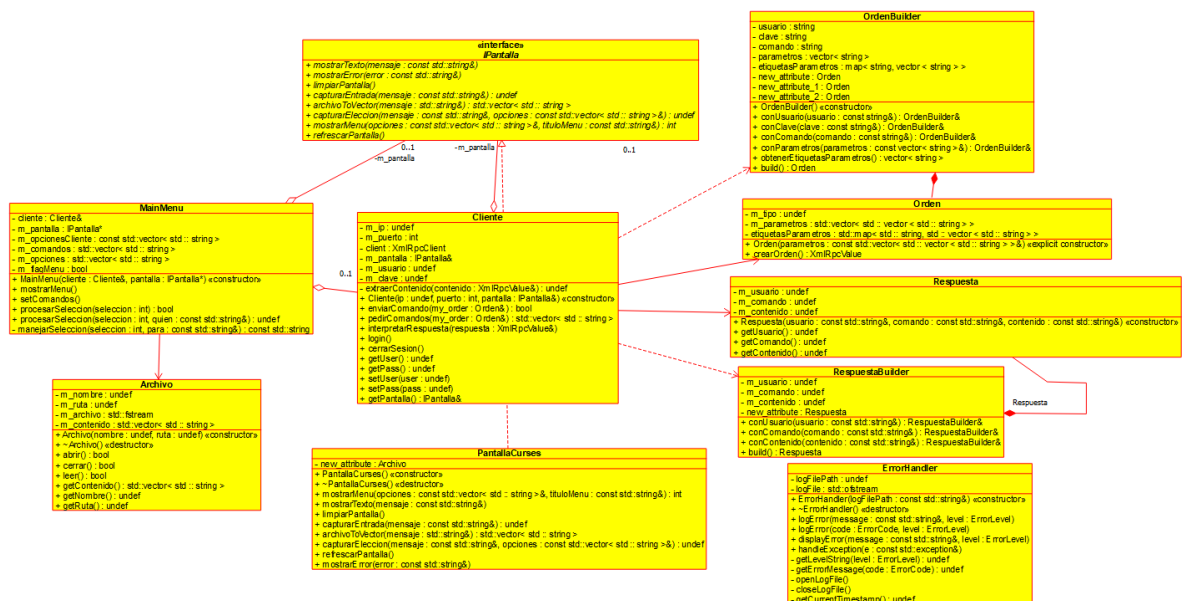


Figura 3: Esquema general del cliente

El diseño modular del sistema permite la separación de responsabilidades en tres capas principales:

- **Capa de Modelo:** Incluye las clases y métodos que representan la lógica del robot y los comandos en G-Code. La clase ‘ControladorRobot’ es responsable de interpretar las instrucciones y coordinar los movimientos del robot. En el caso del cliente esta capa se ve representada por la clase MainMenu, Orden, Respuesta, y sus Builders.
- **Capa de Control:** Gestiona la comunicación entre el cliente y el servidor, y garantiza la correcta ejecución de los comandos. La clase ServidorControl‘ actúa como mediador entre la interfaz de usuario y la lógica interna del robot. Por otro lado en el cliente esta capa se ve representada por la clase Cliente.
- **Capa de Vista:** Corresponde a la interfaz de usuario en el cliente y en el servidor, donde se envían las solicitudes de movimiento y comandos específicos. En el servidor, la clase ‘InterfazConsola’ se encarga de recibir entradas del usuario y enviarlas a la clase que corresponda ControladorRobot, o ServidorControl, y en el Cliente, la clase PantallaCurses se encarga de mostrar la interfaz al usuario, es comandada por la clase MainMenu que le dice qué debe mostrar y cuando.

2.2. Descripción de los componentes software

A continuación se describen los componentes principales del sistema, detallando sus funcionalidades y la interrelación entre ellos para lograr un flujo de comunicación eficiente y estructurado:

- **XML-RPC en C++:** Protocolo que permite la comunicación remota entre el cliente y el servidor. A través de este, se envían comandos desde el cliente hacia el servidor, los cuales son interpretados y ejecutados en tiempo real. La elección de XML-RPC se basó en su simplicidad para gestionar datos en formato XML, lo cual permite su implementación sin una sobrecarga significativa en la red. Se utilizó la librería XMLRPC++ de Chris Morley.
- **Librería PDCurses en C++:** Esta librería es una adaptación de su versión para Linux *ncurses* la cual permite aprovechar todas las características de la ventana de comandos permitiendo hacer interfaces gráficas bastante interactivas similar a la librería *questionary* en Python.
- **Librería serial en Python:** Responsable de la comunicación directa entre el servidor y el robot mediante el puerto serial. Esta librería permite enviar y recibir datos en tiempo real, facilitando el control de los actuadores y sensores del robot. La biblioteca *serial* se configura en el servidor para la transmisión y recepción de comandos G-Code, asegurando un flujo de datos adecuado para la ejecución de movimientos precisos.
- **Librería SimpleXMLRPCServer en Python:** Esta librería ofrece todas las funciones necesarias para manejar de manera sencilla permitiendo abrir un servidor y registrar funciones de manera flexible. Combinando estas características con que el servidor funcione en un hilo aparte se convierte en un servidor robusto.

- **Librería `questionary` en Python:** Esta librería hace más amena la interacción con el usuario permitiéndole a este utilizar un menú selectivo en lugar de un menú común dependiente de la entrada del usuario.
- **G-Code:** Protocolo estándar para el control de movimientos en máquinas CNC y robots. En este proyecto, el G-Code permite especificar posiciones, velocidades y tareas que el robot debe ejecutar. El uso de G-Code se adapta para gestionar instrucciones relativas a cada articulación del robot, maximizando la precisión en la ejecución de movimientos complejos.

2.3. Instalación, vinculación y uso del sistema

El proceso de instalación y configuración de los componentes del sistema es esencial para garantizar el correcto funcionamiento de la aplicación. A continuación se describen los pasos necesarios:

2.3.1. Instalación del servidor

El servidor, desarrollado en Python, requiere la instalación de varias dependencias:

- `xmlrpc.server`: Implementa la funcionalidad de servidor XML-RPC.
- `serial`: Biblioteca para comunicación serial con el hardware del robot.
- `logging`: Registra eventos y errores para facilitar el análisis y depuración.

Para instalar estas dependencias, se debe ejecutar el siguiente comando en la terminal:

```
pip install -r requirements.txt
```

Una vez instaladas, el servidor se puede iniciar mediante:

```
python main.py
```

```
proyecto_integrador\codigos\servidor\codigo
```

2.3.2. Instalación del cliente

Para la compilación del cliente debe ejecutarse el comando "make"^{en} una terminal de comandos sobre la carpeta "bins" del directorio del cliente. Esto usará el Makefile allí presente, si se desea puede utilizarse "make clean" para limpiar el ejecutable anterior.

2.4. Diagrama de Secuencia

El diagrama de secuencia a continuación detalla el proceso de comunicación entre el cliente y el servidor, desde el envío de un comando hasta la recepción de la respuesta por parte del robot.

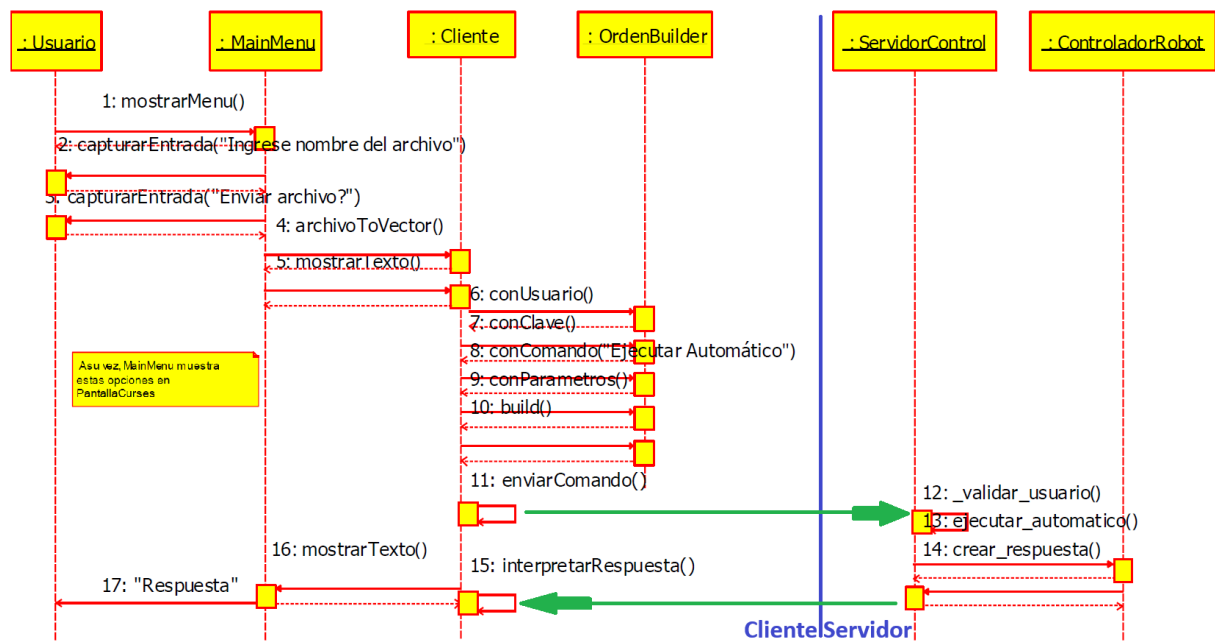


Figura 4: Diagrama de secuencia: Ejecutar Comando con Archivo G-Code

2.5. Diagrama de Actividad

Este diagrama muestra el ciclo completo de funcionamiento del servidor, desde que recibe una solicitud hasta que la procesa y responde al cliente.

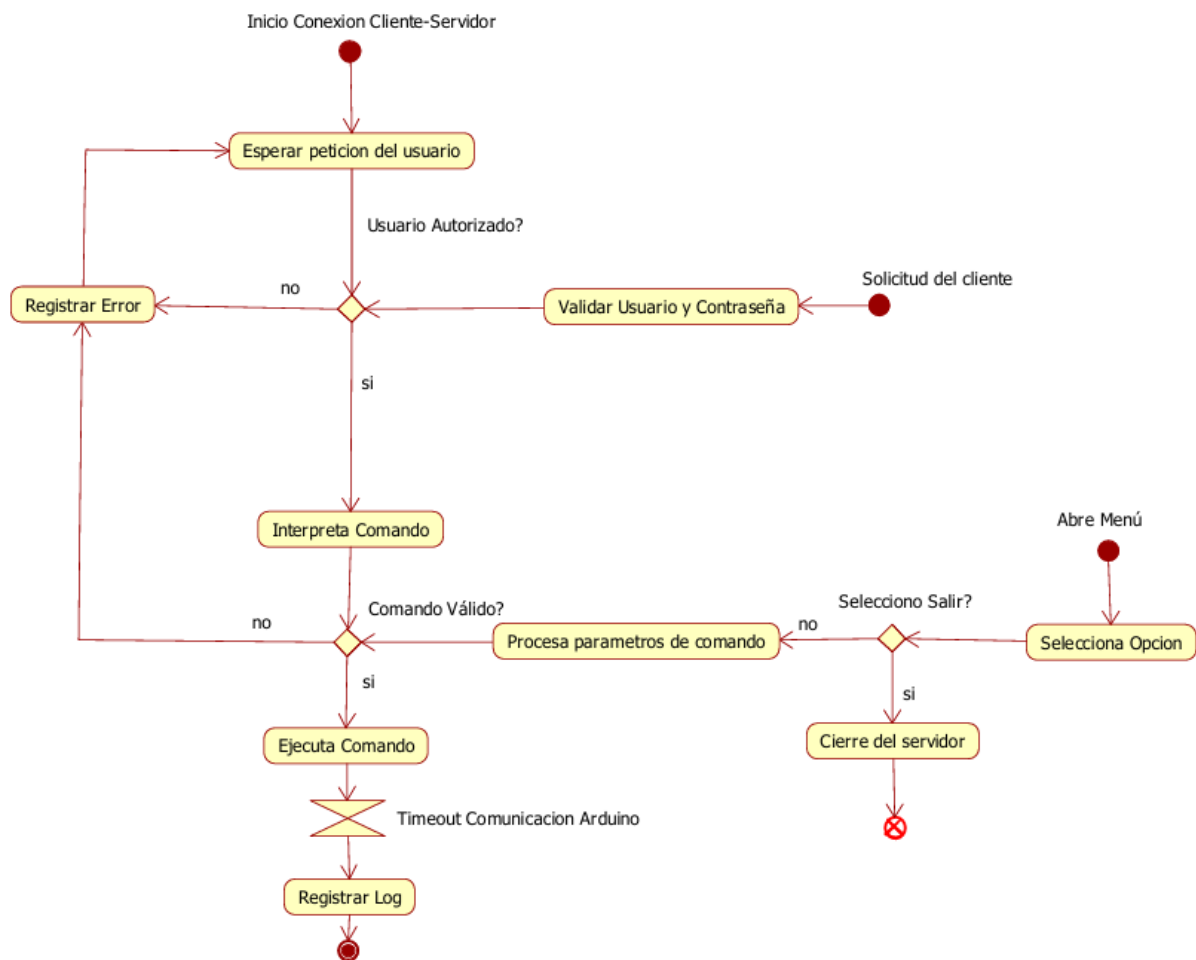


Figura 5: Diagrama de actividad general

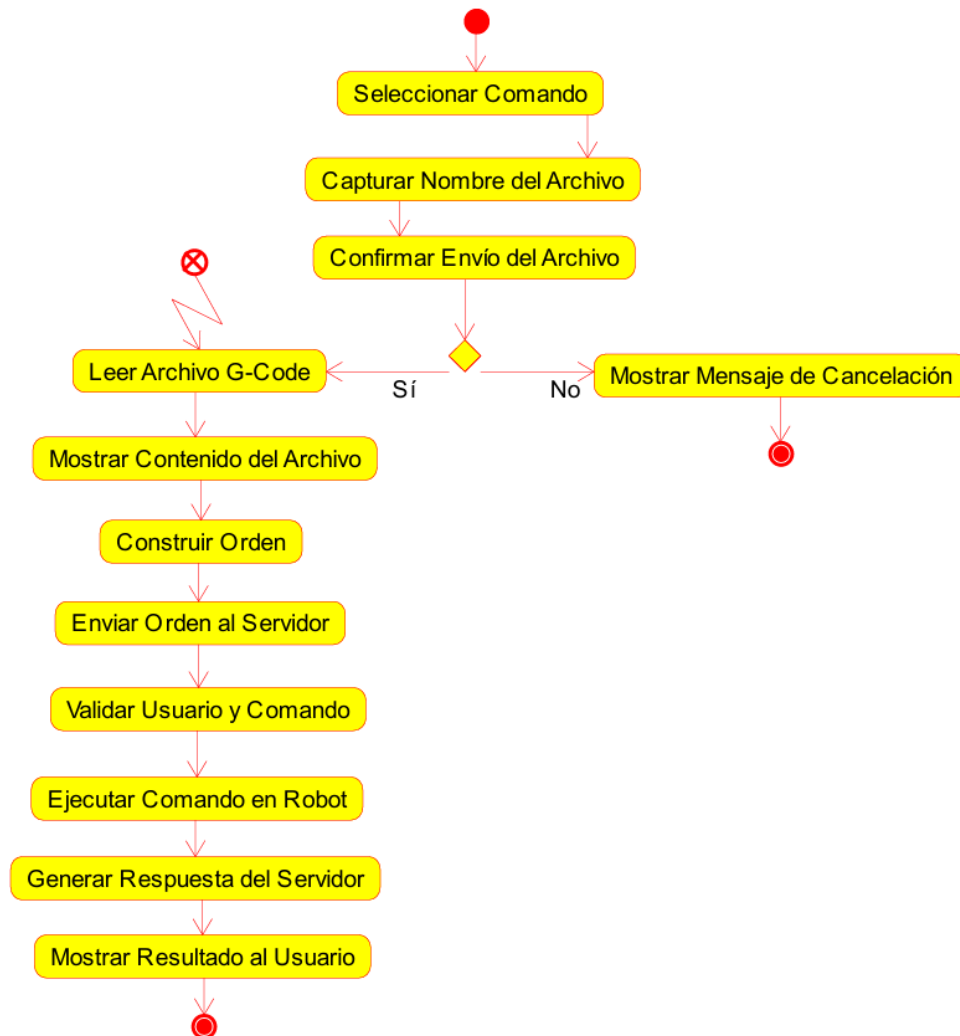


Figura 6: Diagrama de actividad: Ejecución de Comando G-Code

2.6. Detalles de instalación, configuración y ejecución

Aspectos técnicos clave para la instalación y ejecución del sistema:

- Configuración del servidor para recibir conexiones XML-RPC en el puerto especificado (9000 en la implementación).
- Acceso del cliente a la IP del servidor, configurada para permitir la comunicación remota.

2.7. Capturas de Pantalla

Las siguientes capturas ilustran la ejecución del sistema tanto en el servidor como en el cliente, mostrando las interfaces para la Ejecución Automática desde el mismo.

```

? Seleccione una opción: (Use arrow keys)
» Conectar al Robot
  Iniciar Servidor RPC
  Mostrar las últimas 100 líneas del Log (Admin)
  Mostrar Ayuda
  Actualizar
  Salir

```

Figura 7: Interfaz inicial del servidor

```

? Seleccione una opción: (Use arrow keys)
» Desconectar del Robot
  Listar Comandos Disponibles
  Mostrar/Editar Parámetros de Conexión
  Cambiar Modo de Trabajo (Actual: manual)
  Cambiar a Modo Relativo
  Desactivar Motores
  Mover Efector Final (con Velocidad)
  Mover Efector Final (solo Posición)
  Homming
  Aprendizaje (On/Off)
  Ejecución Automática
  Reportar Estado
  Reportar Posición Actual
  Activar Efector
  Detener Servidor RPC
  Mostrar las últimas 100 líneas del Log (Admin)
  Mostrar Ayuda
  Actualizar
  Salir

```

Figura 8: Interfaz del servidor con robot ya conectado

```

Conectar al Robot,127.0.0.1,consola_local,fallido,2024-11-05 19:24:37
iniciar_servidor_rpc,127.0.0.1,consola_local,exitoso,2024-11-05 19:24:38
Conectar al Robot,127.0.0.1,consola_local,exitoso,2024-11-05 19:25:27
Activar Motores,127.0.0.1,consola_local,exitoso,2024-11-05 19:25:55
Homming,127.0.0.1,consola_local,exitoso,2024-11-05 19:26:04
? Seleccione una opción: (Use arrow keys)
» Desconectar del Robot
  Listar Comandos Disponibles
  Mostrar/Editar Parámetros de Conexión
  Cambiar a Modo Relativo
  Desactivar Motores
  Mover Efector Final
  Homming
  Aprendizaje (On/Off)
  Ejecución Automática
  Reportar Estado
  Reportar Posición Actual
  Activar Efector
  Detener Servidor RPC
  Mostrar las últimas 100 líneas del Log (Admin)
  Mostrar Ayuda
  Actualizar
  Salir

```

Figura 9: Mostrar últimas 100 líneas del Log



Figura 10: Interfaz inicial del cliente

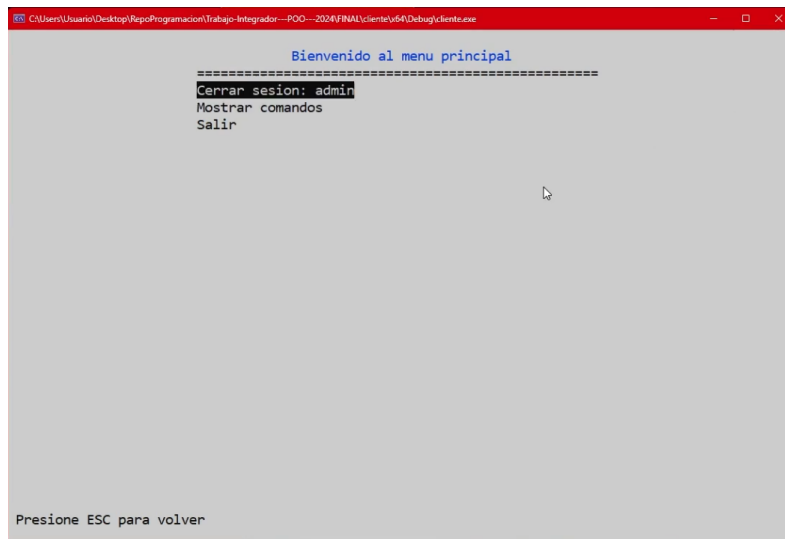


Figura 11: Sesión iniciada en cliente

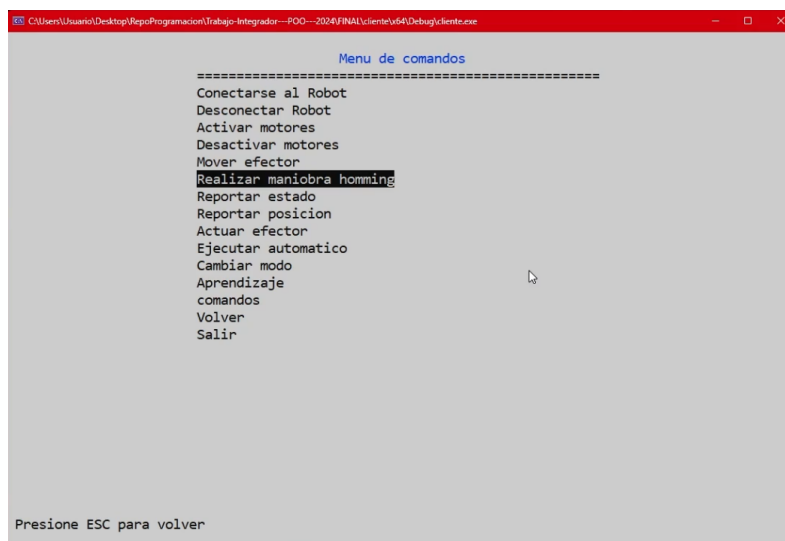


Figura 12: Menú de comandos del cliente

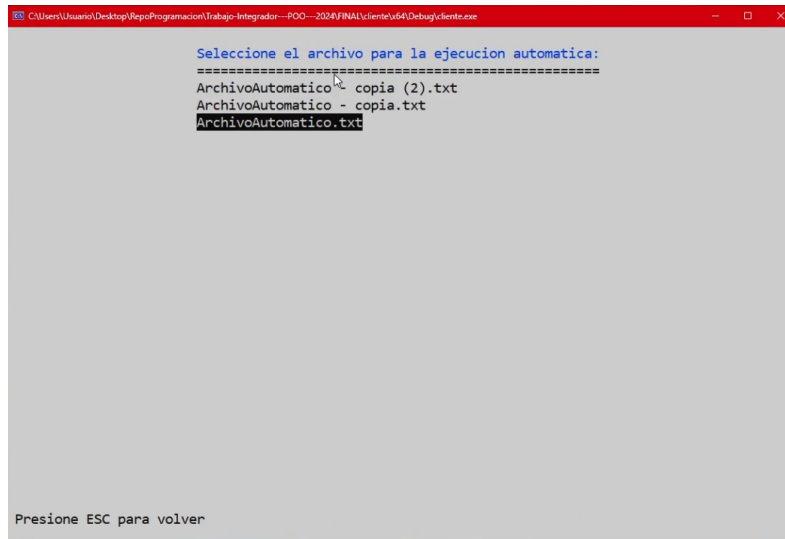


Figura 13: Selección Archivo para Ejecución Automática en cliente

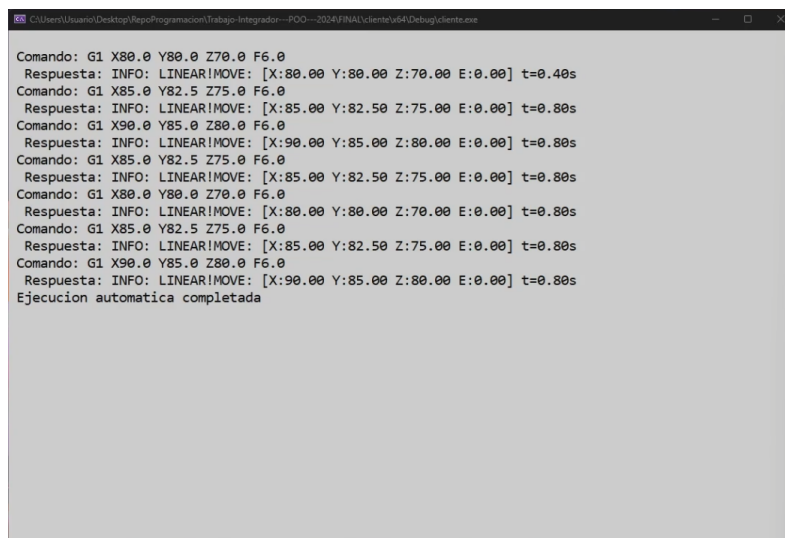


Figura 14: Respuesta final en cliente tras Ejecución Automática

2.8. Pruebas Unitarias

Se realizaron pruebas unitarias de los *metodos* del módulo *GestorDeArchivos*. Aun este siendo el más sencillo de los implementados, tiene un papel crucial en la autenticación de usuario y en el modo de ejecución automático. Se utilizo **pytest** para la realización de cada prueba, en adicional de la libreria por defecto **unittest** particularmente el modulo **mock**. La eleccion de **pytest** por sobre **unittest** esta motivada unicamente por una leve superioridad en términos de facilidad de sintaxis y flexibilidad al momento de preparar el contenido previo para probar cada metodo, gracias a los *fixtures* que ofrece **pytest**. Para correr las pruebas unitarias debe ejecutarse el comando **pytest** en el directorio correspondiente.

```

import pytest
from unittest.mock import patch, mock_open
from GestorDeArchivos import GestorDeArchivos
import json

@pytest.fixture
def gestor_archivos():
    # Fixture para crear un gestor de archivos con un nombre de archivo ficticio
    return GestorDeArchivos("archivo_test.txt")

def test_limpiar_archivo(gestor_archivos):
    # Probamos que el método limpiar_archivo vacíe el archivo
    with patch("builtins.open", mock_open()) as mock_file:
        gestor_archivos.limpiar_archivo()
        mock_file.assert_called_once_with("archivo_test.txt", "w")

def test_guardar_linea(gestor_archivos):
    # Probamos que el método guardar_linea guarde correctamente una línea
    objeto_json = json.dumps({"clave": "valor"})
    with patch("builtins.open", mock_open()) as mock_file:
        gestor_archivos.guardar_linea(objeto_json)
        mock_file.assert_called_once_with("archivo_test.txt", "a")
        mock_file().write.assert_called_once_with(objeto_json + "\n")

def test_leer_contenido(gestor_archivos):
    # Probamos que el método leer_contenido lea y deserialice el contenido JSON
    contenido = '{"clave": "valor"}\n{"clave": "otro valor"}\n'
    with patch("builtins.open", mock_open(read_data=contenido)) as mock_file:
        resultado = gestor_archivos.leer_contenido()
        mock_file.assert_called_once_with("archivo_test.txt", "r")
        assert resultado == [{"clave": "valor"}, {"clave": "otro valor"}]

```

Figura 15: Algunos de los métodos testeados. En la imagen: limpiar archivo; guardar línea; leer contenido

3. Disposición de directorios

```

a requirements.txt
a
a settings.json
a
+---archivos
a     archivoRecibido.txt
a     log_trabajo.csv
a     ordenes_ejecutadas.txt
a     ordenes_solicitadas.txt
a     usuarios.json
a
+---codigo
a     ControladorRobot.py
a     GestorDeArchivos.py
a     GestorDeArchivos_test.py
a     InterfazConsola.py
a     Logger.py
a     main.py
a     ManejadorErrores.py
a     ServidorControl.py
a

```

Figura 16: Arbol de directorio del servidor

```

codigos
└─ cliente
   └─ bins
      ├── archivos_gcode
      ├── Makefile
      └── cliente.exe
   └─ inc
      ├── Orden.h
      ├── OrdenBuilder.h
      ├── RespuestaBuilder.h
      ├── Respuesta.h
      ├── MainMenu.h
      ├── IPantalla.h
      ├── PantallaCurses.h
      ├── Cliente.h
      ├── Archivo.h
      └── ErrorHandler.h
   └─ src
      ├── main.cpp
      ├── Cliente.cpp
      ├── Archivo.cpp
      ├── ErrorHandler.cpp
      ├── MainMenu.cpp
      └── PantallaCurses.cpp
   ├── README.md
   └── CMakeLists.txt

```

Figura 17: Arbol de directorio del cliente

4. Conclusiones

4.1. Dificultades enfrentadas

Durante el desarrollo del proyecto, se presentaron diversos desafíos técnicos. Uno de los principales problemas fue la implementación correcta de la comunicación cliente-servidor mediante XML-RPC y la sincronización de los comandos enviados con el hardware del robot, aspectos fundamentales para garantizar un control preciso y confiable en tiempo real. Otro reto importante fue el manejo del dinamismo en la interfaz del cliente, ya que inicialmente todas las opciones de control estaban predefinidas en el cliente, lo que restringía la flexibilidad y hacía difícil adaptar nuevas funcionalidades o comandos desde el servidor.

Aunque se analizaron varias estrategias para superar estos obstáculos, como el uso de técnicas de manejo de excepciones y pruebas iterativas para ajustar la comunicación, en esta etapa del proyecto no se lograron implementar de manera completa. Esto dejó planteada una posible extensión futura para mejorar la flexibilidad y control desde el servidor, facilitando el despliegue de cambios en las opciones del cliente sin modificaciones directas en su código.

Sin dudas otra de las complicaciones encontradas fue la de aferrarse constantemente a una orientación a objetos, la cual por momentos presentaba ciertas redundancias y extensiones que podían verse innecesarias, pero para ciertas secciones del código fue fundamental el estar trabajando con este enfoque.

Luego por el lado del hardware se encontraron dificultades al momento de comunicarse con el driver presente en el Arduino, el cual al tener tiempos aleatorios de respuesta para cada movimiento, el servidor debía tener cierto delay para esperar la respuesta, ya que sino, el servidor entendía que la orden no se había ejecutado correctamente.

4.2. Bugs

Actualmente, existen errores visuales en la interfaz del cliente debido a limitaciones en su librería. Estos errores se manifiestan al redimensionar la pantalla, lo que impide una correcta actualización de la visualización, llegando a mostrar algunos textos y botones de manera desordenada.

4.3. Ventajas y desventajas de los componentes utilizados

Las bibliotecas utilizadas, como `XMLRPC++` y `serial`, ofrecieron ventajas en cuanto a la simplicidad de implementación. Sin embargo, se detectaron limitaciones en cuanto a la escalabilidad de la arquitectura para aplicaciones de mayor complejidad. En cuanto a demás desventajas, sin duda `PDCurses` presentó dificultades pero no para su uso, sino que para la etapa de debug del código, esta librería, aun con su increíble virtud de hacer de la CLI una interfaz muchísimo más amena para el usuario, toma control sobre la consola y no permitía de forma sencilla utilizar `iostream` para ello debía primero terminarse la pantalla de `PDCurses` para luego utilizar `cout`, cuestión que traía complicaciones. Aun así, destacamos las increíbles capacidades de esta librería que ninguno de los tres conocía.

4.4. Posibles extensiones

El sistema podría extenderse considerablemente mediante la implementación de una interfaz gráfica para el cliente, lo que facilitaría el control del robot de forma más intuitiva y accesible para el usuario. También sería ideal que el servidor gestionara completamente la interfaz del cliente y sus funciones. En este esquema, al iniciar sesión, el servidor enviaría las opciones disponibles al cliente, quien se limitaría a renderizar la interfaz y responder según las instrucciones recibidas. Esto centralizaría el control en el servidor, permitiendo adaptar o modificar las funciones del cliente sin necesidad de realizar cambios directos en su código, lo cual incrementaría la escalabilidad y la capacidad de actualización del sistema.

Además, se podrían agregar funcionalidades avanzadas al sistema de control mediante G-Code, soportando movimientos más complejos y precisos del robot. Estas extensiones permitirían un manejo más dinámico y adaptable del sistema, tanto en términos de funcionalidad como de control de usuario. Por último, el servidor debería poder enviar absolutamente toda la información necesaria para generar el menú y sus respectivas opciones internas y parámetros de cada comando.

5. Referencias

5.1. Bibliografía y sitios web consultados

- Aranda, C. O. (2024). Programación Orientada a Objetos - Guía de Trabajos Prácticos.
- Morley, C. (2003). Documentación de XMLRPC++. Disponible en: <https://xmlrpcpp.sourceforge.net/>.

5.2. Recursos adicionales consultados

- Referencia de G-Code. Disponible en: <http://example.com/gcode>.
- Tutorial sobre la librería serial en Python. Disponible en: <http://example.com/serial-python>.